

PROJECT PROPOSAL

GROUP PROJECT

IIT271-2

Group No: IIT-18

UniAlloc

**Human Resource Allocation System For
University Office**

Industrial Information Technology Degree Program

Department of Computer Science and Informatics

Faculty of Applied Sciences

Uva Wellassa University of Sri Lanka

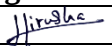
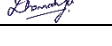
2026

DECLARATION

We hereby declare that the project will be developed by us and will be our own effort and that no part will be plagiarized without citations under the supervision of Ms. Dr. N. P. Samarasinghe, Senior Lecturer, Department of Computer Science and Informatics, Faculty of Applied Sciences, Uva Wellassa University, Badulla, Sri Lanka. This Project Proposal is submitted for the partial fulfillment of the requirement of the course unit IIT271-2, Project I for the degree of Industrial Information Technology Degree Program.

GROUP DETAILS

Group No: IIT18

No.	Name of the Student	Index Number	E-mail address	Signature
1	Mr. Suhan W.H.	UWU/IIT/23/011	iit23011@std.uwu.ac.lk	
2	Mr. Sandeep G.C.	UWU/IIT/23/020	iit23020@std.uwu.ac.lk	
3	Mr. Chathuranga D.L.	UWU/IIT/23/030	iit23030@std.uwu.ac.lk	
4	Mr. Arachchi D.C.K.	UWU/IIT/23/024	iit23024@std.uwu.ac.lk	

SUPERVISOR DETAILS

Name of the Supervisor	E-mail	Contact Number
Dr. N. P. Samarasinghe	nayanathara@uwu.ac.lk	+94 553560090

Name of the Co-Supervisor	E-mail	Contact Number
Mr. K.B.L. Jayawickrama	kavindulakshan2980@gmail.com	+94 704512641

Project Approval:

.....

Dr. N. P. Samarasinghe

Date: 19.052026

.....

Mr. K.B.L Jayawickrama

Date: 19.052026

Table of Contents

CHAPTER 1: INTRODUCTION	1
Title of the Project	1
Project Description.....	1
Background and Motivation	2
Problem in Brief.....	2
Proposed Solution	2
Project Aim and Objectives	4
Assumptions and Constraints.....	4
CHAPTER 2: METHODOLOGY	5
Waterfall Phases.....	5
2.1. Requirements Gathering and Analysis.....	6
2.2. System Design (Architecture, Modules, Data Flow, Security, UI/UX).....	7
2.3 Implementation	10
2.4 Testing and Debugging	11
2.5. Deployment.....	12
2.6. Maintenance	12
CHAPTER 3: REQUIREMENTS IDENTIFICATION	14
Functional Requirements	14
Non-Functional Requirements	15
System Requirements (Hardware/Software).....	16
User Roles	17
CHAPTER 4: PROJECT PLAN	18
Gantt Chart.....	18
References	19

List of Tables

Table 1: Project Plan (Gantt Chart)

List of Figures

Figure 01: UniAlloc Platform

Figures 02: Waterfall Model

Figures 03: Three-Tier Architecture

Figures 04: UI/UX Concept

Figures 05: System Implementation plan

Figures 06: Maintenance

CHAPTER 1: INTRODUCTION

Title of the Project

UniAlloc —Human Resource Allocation System For University Office

Project Description

UniAlloc is a web-based Human Resource Allocation System designed for a university Dean's Office. It replaces ad-hoc, spreadsheet-driven workload management with a transparent, role-based platform that enables Deans and Department Heads to allocate assignments based on real-time lecturer capacity, supports cross-department and cross-faculty request workflows, and provides a formal portal for students to request project/thesis supervisors with trackable status updates.

This system focuses on allocating and tracking academic work across a two-tier university structure (Faculties containing Departments). It supports assignment creation, workload/capacity tracking, progress updates, cross-boundary request workflows, and student supervision requests. This proposal covers requirements analysis, design, implementation approach, and evaluation for a web-based prototype.

Project Boundaries:

- Role-based access for System Admin, Dean, Department Head, Lecturer, and Student
- Faculty/Department management (faculty contains departments; users belong to departments)
- Assignment management (create, prioritize, estimate hours, set deadlines, assign, track status)
- Real-time workload capacity view (assigned hours vs. available hours) and overload warnings
- Cross-department and cross-faculty request workflows for out-of-scope allocations
- Progress tracking (percentage updates with timestamped notes)
- Student supervisor request portal with status tracking and Dean review/assignment
- Notifications and audit logs for critical system actions

Background and Motivation

University Dean's Offices and Departments often manage lecturer workloads and supervision assignments using spreadsheets, informal messaging, and personal judgement. This leads to limited visibility into who is available, inconsistent allocation of decisions, duplicated commitments, and difficulty tracking progress across multiple departments and faculties.

This project is motivated by the need for a centralized, hierarchy-aware platform that provides real-time capacity information before assignments are made, supports formal cross-department/cross-faculty request workflows, and maintains a clear audit trail of who assigned what work and when. By combining allocation, monitoring, and structured requests in one system, UniAlloc aims to improve fairness, accountability, and decision-making in academic HR management.[11],[12]

Problem in Brief

- **Unbalanced lecturer workloads:** Without a centralized capacity view, some lecturers become overloaded while others remain underutilized.
- **Informal cross-department assignments:** When work needs to cross department/faculty boundaries, there is no formal channel, causing confusion and disputes.
- **Unsystematic student project requests:** Supervisor requests are often handled informally with little tracking, leading to lost requests and inconsistent outcomes.

Proposed Solution

UniAlloc provides a centralized platform where Deans and Department Heads create assignments and allocate them to lecturers based on real-time capacity information (assigned hours vs. available hours). When a required lecturer is outside the assigner's authority scope, the system supports formal cross-department or cross-faculty requests that are tracked to completion. Lecturers update progress on assigned work, and students submit supervisor requests through a structured portal that routes to the Dean for review and assignment.[12]

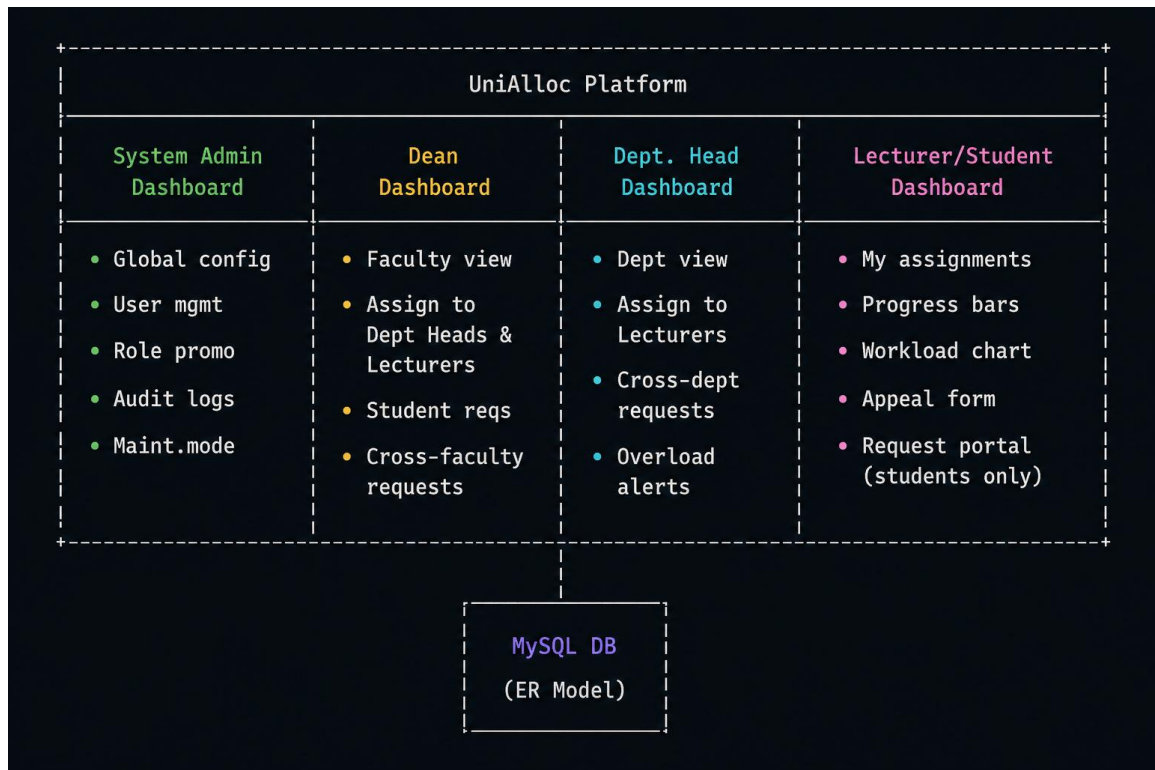


Figure 01: UniAlloc Platform

How it works:

1. Deans and Department Heads create assignments with priority, deadline, and estimated hours.
2. Before assigning, the system shows each lecturer's capacity: assigned hours vs. available hours.
3. If the needed lecturer is out of scope, the assigner submits a cross-department or cross-faculty request and tracks its status.
4. Lecturers update progress (0–100%) with timestamped notes; dashboards reflect live status.
5. Students submit supervisor requests; the Dean reviews and assigns an appropriate lecturer/department head.
6. Overload detection triggers notifications and helps supervisors identify underloaded alternatives.

Project Aim and Objectives

Aim

To design and implement a role-based, hierarchy-aware web system for a university faculty Office that enables fair and transparent lecturer workload and supervision allocation through real-time capacity visibility, structured request workflows, and progress tracking.

Objectives

- Implement five-level role-based access control (System Admin, Dean, Department Head, Lecturer, Student).
- Provide assignment management with priority, deadlines, and estimated hours.
- Implement real-time workload capacity tracking (assigned hours vs. available hours) with overload alerts.
- Support cross-department and cross-faculty request workflows for out-of-scope allocations.
- Provide per-assignment progress tracking and role-based dashboards for monitoring.
- Provide a student portal for supervisor requests with status tracking and Dean review.
- Ensure that the system is secure, usable, maintainable, and scalable for institutional use.

Assumptions and Constraints

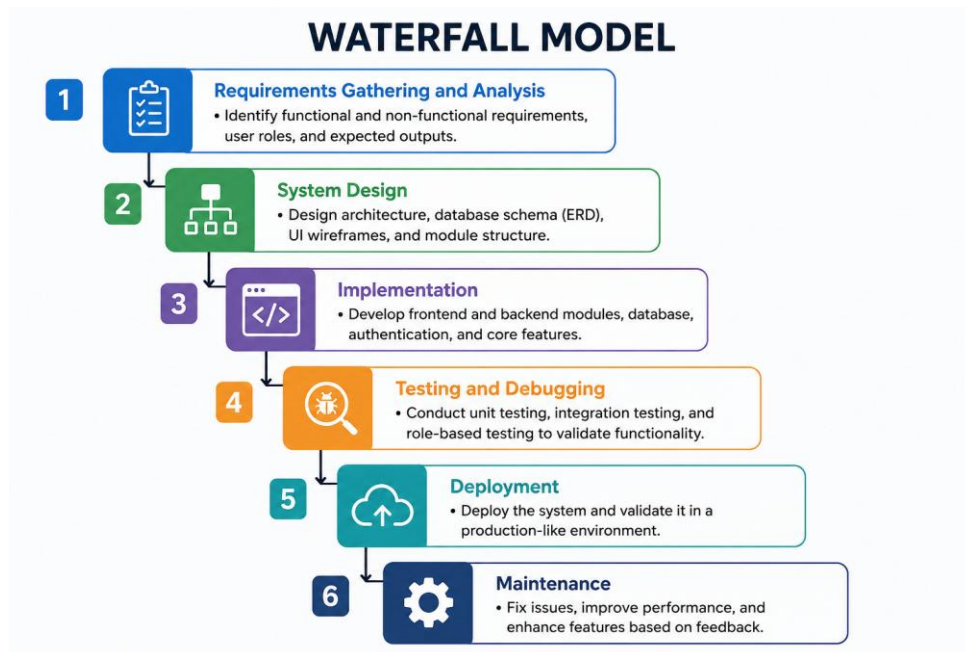
- **Assumption:** Each lecturer has a defined capacity (e.g., weekly available hours) that can be configured by authorized administrators.
- **Assumption:** Assignment effort is estimated in hours and used for workload calculations and overload detection.
- **Constraint:** Authority scope is enforced: Deans operate within their faculty; Department Heads operate within their department; cross-boundary allocations require formal requests.
- **Constraint:** The initial prototype focuses on core allocation, tracking, and request workflows; advanced integrations and optimization are deferred to later phases.

CHAPTER 2: METHODOLOGY

This project follows the Waterfall Software Development Methodology. Waterfall is a sequential approach where each phase is completed before moving to the next phase. This methodology is suitable when requirements are relatively well understood, and the project requires structured documentation and clear deliverables.[1],[2]

Waterfall Phases

1. **Requirements Gathering and Analysis:** Identify functional and non-functional requirements, user roles, and expected outputs.
2. **System Design:** Design architecture, database schema (ERD), UI wireframes, and module structure.
3. **Implementation:** Develop frontend and backend modules, database, authentication, and core features.
4. **Testing and Debugging:** Conduct unit testing, integration testing, and role-based testing to validate functionality.
5. **Deployment:** Deploy the system and validate it in a production-like environment.
6. **Maintenance:** Fix issues, improve performance, and enhance features based on feedback.



Figures 02: Waterfall Model

2.1. Requirements Gathering and Analysis

This phase focuses on understanding the Dean's Office workload allocation process, identifying stakeholders, capturing business rules, and translating them into clear functional and non-functional requirements. The outcome is a validated requirements baseline that guides design and implementation while minimizing rework and scope creep.[1]

Key activities:

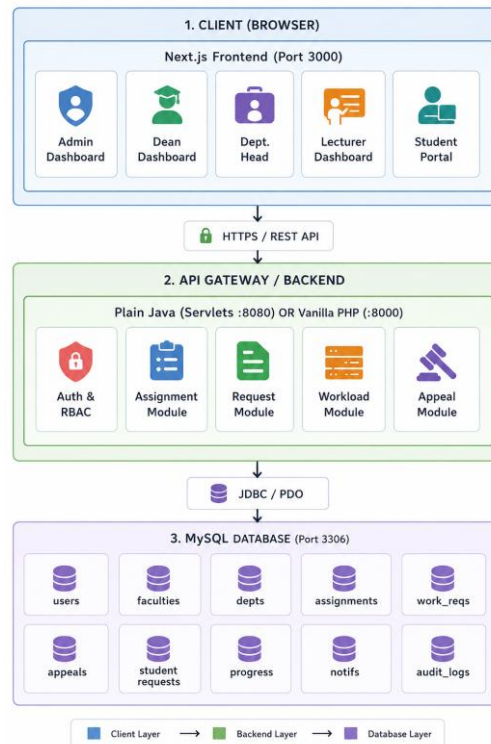
- Identify stakeholders (System Admin, Dean, Department Head, Lecturer, Student) and their responsibilities.
- Conduct requirement elicitation through interviews, observation, and review of existing workflows (spreadsheets, forms, informal approvals).
- Define system boundaries and scope (in-scope and out-of-scope features).
- Document functional requirements (use cases such as allocation, overload detection, request workflows, progress updates).
- Document non-functional requirements (security, usability, performance, availability, auditability).
- Create early models such as user-role summaries and process flow summaries.

Deliverables:

- Requirements Specification (Functional + Non-Functional Requirements)
- User role definitions and permissions overview
- Initial acceptance criteria for core functions
- Assumptions/constraints list and risk notes

2.2. System Design (Architecture, Modules, Data Flow, Security, UI/UX)

During the System Design phase, the project defines the overall solution blueprint, including the application architecture, major modules, high-level data flow, security controls, and UI/UX approach. This ensures the implementation phase proceeds with clear technical guidance and consistent design decisions.[1],[4],[7]



Figures 03: Three-Tier Architecture

System Architecture

The system is designed using a standard three-tier architecture consisting of a client layer (web browser/Next.js web app), application layer (pure REST API built with Vanilla PHP), and data layer (MySQL database). The frontend provides role-based dashboards and user interfaces, while the backend exposes JSON-only endpoints for authentication, role-based access control, and business logic for assignments, capacity, requests, notifications, and reporting. The backend communicates with MySQL using PDO with parameterized queries and returns consistent JSON responses with appropriate HTTP status codes.

Major System Modules

- **Authentication & RBAC Module:** handles login, token/session validation, and strict role/scope enforcement.
- **User, Faculty & Department Module:** manages faculties, departments, users, capacity hours, and authority scope.
- **Assignment Module:** assignment creation, prioritization, deadlines/effort hours, and status tracking.
- **Capacity & Overload Module:** calculates assigned vs. available hours and triggers overload alerts.
- **Request & Appeal Module:** cross-department/cross-faculty requests and workload appeals workflows.
- **Student Request Module:** supervision request submission, Dean review, assignment, and status tracking.
- **Notifications Module:** alerts for assignments, deadlines, overload conditions, and request status changes.
- **Audit Logging Module:** records critical actions for accountability and traceability.

High-Level Data Flow (Summary)

1. A user (System Admin/Dean/Department Head/Lecturer/Student) authenticates to the system and receives a secure session/token.
2. System Admin manages global settings; Deans manage faculty-scoped users and structure as permitted.
3. Deans and Department Heads create assignments (priority, deadline, estimated hours) within their authority scope.
4. Before assigning, the backend calculates lecturer capacity (assigned hours vs. available hours) and flags overload risk.
5. If the target lecturer is outside scope, the assigner submits a cross-department/cross-faculty request; the approver reviews and updates status.
6. Allocated lecturers update progress (0–100%) with timestamped notes; updates are stored in the progress log.
7. Students submit supervisor requests; requests route to the Dean for review/assignment, and students can track status changes.

8. Notifications are generated for key events, and critical actions are recorded in audit logs.
9. Role-based dashboards aggregate workload, capacity, progress, and pending requests for monitoring and decision-making.

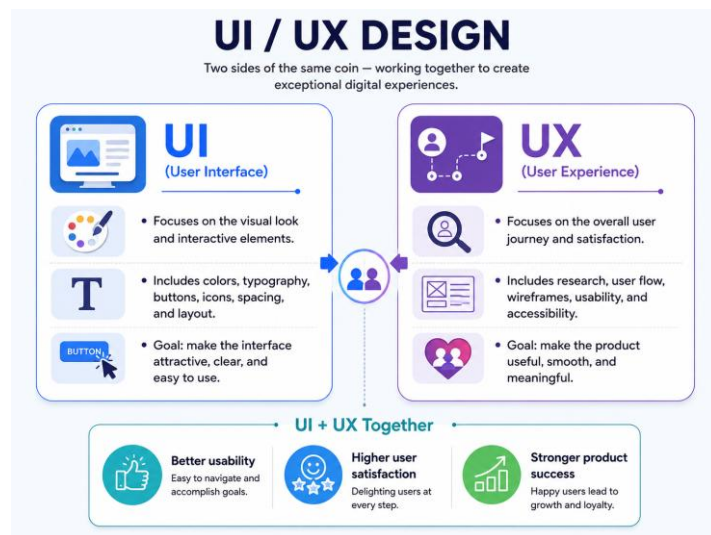
Security Considerations (Design Summary)

- **Authentication:** Use token-based authentication (e.g., JWT) with expiry and secure credential handling.
- **Authorization:** Enforce role-based access checks on every protected action to prevent broken access control.
- **Input validation:** Validate/sanitize input and use parameterized queries to reduce injection risks.
- **Logging and monitoring:** Maintain audit logs for key actions (logins, assignments, approvals, role changes).
- **Secure configuration:** Apply secure defaults and keep components updated, aligning with OWASP Top 10 guidance.

UI/UX Design Philosophy

The user interface is designed to be clean and task-focused, allowing users to quickly understand workload, task status, and progress. The design emphasizes readability, consistency, and role-based visibility of functions. [3],[5],[7]

- Dark-first visual theme to reduce eye strain and make dashboards comfortable for long sessions.
- Glass morphism-inspired surfaces (soft blur/translucent cards) to separate content layers while keeping the UI modern.
- Gradient accent colors for emphasis (primary actions, status highlights, and key metrics) without overwhelming core data.
- Micro-animations for feedback and clarity (hover states, transitions, progress changes) where applicable.
- Consistent layout, labels, and navigation across role-based dashboards to reduce learning effort.
- Responsive design for desktop and mobile devices, maintaining accessibility (contrast, readable sizes, and clear status indicators).



Figures 04: UI/UX Concept

2.3 Implementation

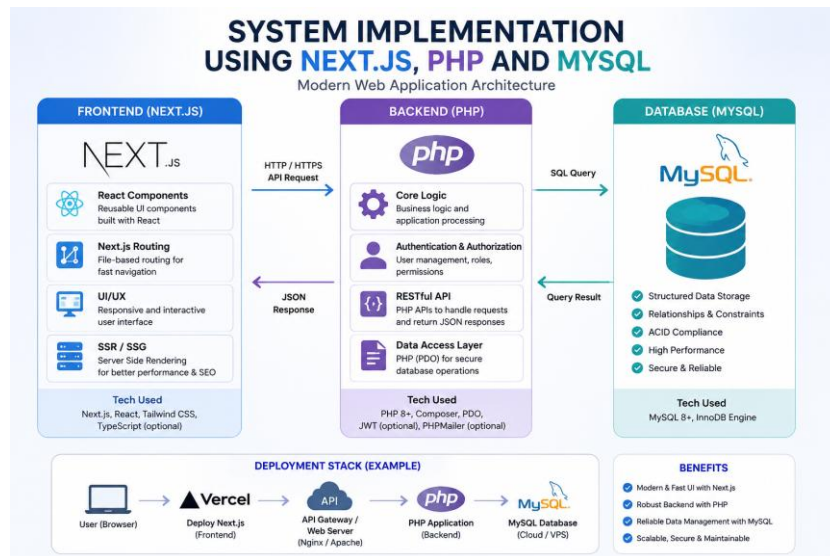
In this phase, the system is developed according to the approved design. The implementation includes building the frontend (Next.js) and backend (Vanilla PHP REST API) modules, integrating with the MySQL database, and ensuring role-based access control is enforced across all endpoints and UI screens.[9],[10]

Key activities:

- Set up project repositories, folder structure, and coding standards.
- Implement database schema and seed data for initial testing.
- Develop backend REST APIs for authentication, users, departments, assignments, requests, progress tracking, notifications, and audit logs.
- Implement frontend pages and dashboards aligned with user roles (Admin/Dean/Head/Lecturer/Student).
- Integrate frontend with backend using secure API communication and consistent JSON responses.
- Apply validation rules, error handling, and logging for reliable operation.

Deliverables:

- Working prototype with core modules
- API documentation (endpoints, request/response formats, status codes)
- Integrated frontend dashboards and workflows
- Initial test-ready build



Figures 05: System Implementation plan

2.4 Testing and Debugging

Testing ensures that the developed system meets the stated requirements and works reliably for all user roles. This phase includes validating each module individually and verifying that integrated workflows (e.g., cross-department requests, supervisor assignment) work end-to-end without defects.

Key activities:

- Unit testing for critical backend functions (authentication, authorization checks, workload calculations).
- Integration testing for connected workflows (assignment → capacity check → notification → progress updates).
- Role-based testing to ensure access control rules are correct (only authorized roles can perform actions).
- UI testing for navigation, forms, validation messages, and responsiveness.
- Bug fixing and regression testing after corrections.
- Performance sanity checks (dashboard load time, database query efficiency).

Deliverables:

- Test cases and test results summary
- Bug list and resolution log
- Verified release candidate for deployment
- Updated documentation reflecting final behavior

2.5. Deployment

Deployment involves preparing the system to run in a production-like environment. The application is configured on a web server with the required runtime and database, then verified through smoke tests to ensure that key functions work correctly after deployment.[8]

Key activities:

- Configure server environment (Apache/Nginx, PHP, MySQL, Node.js for frontend build).
- Set up environment variables (database credentials, secret keys, allowed origins).
- Deploy backend APIs and import database schema.
- Build and deploy frontend application bundle.
- Configure security settings (HTTPS, secure headers, access rules).
- Conduct smoke testing and verify login, dashboards, and workflows.
- Create backup/restore procedures for database.

Deliverables:

- Deployed prototype system
- Deployment guide (setup steps, configuration values, troubleshooting tips)
- Verified working environment and baseline backup

2.6. Maintenance

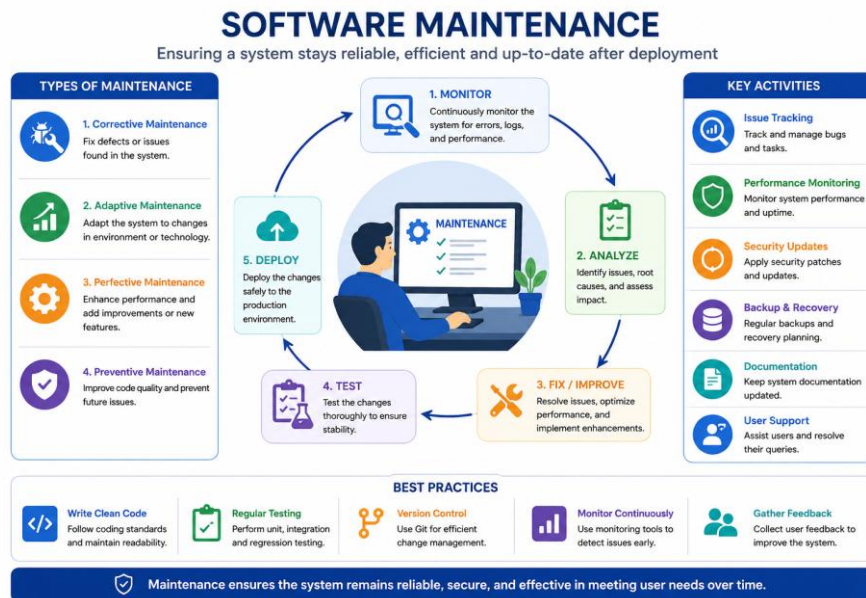
Maintenance begins after deployment and continues throughout the system's usage. The focus is on resolving issues, improving performance, refining usability, and implementing enhancements based on feedback. This phase ensures the system remains stable, secure, and aligned with evolving university needs.[8]

Key activities:

- Monitor system logs and address runtime errors or user-reported issues.
- Apply security patches and dependency updates when required.
- Optimize database queries and improve response times as usage grows.
- Improve UI/UX based on user feedback (navigation, forms, dashboards).
- Add small enhancements such as new reports, filters, or notifications.
- Perform periodic backups and validate restore procedures.

Deliverables:

- Maintenance log (bugs fixed, updates applied)
- Minor feature enhancements and UI refinements
- Updated system documentation and release notes



Figures 06: Maintenance

CHAPTER 3: REQUIREMENTS IDENTIFICATION

Functional Requirements

- **Authentication and Authorization:** The system shall provide secure login and enforce role-based access control (System Admin, Dean, Department Head, Lecturer, Student).
- **Faculty/Department and User Management:** Authorized roles shall be able to manage faculty/department structures and create/manage users within their scope.
- **Assignment Management:** Deans and Department Heads shall be able to create assignments with priority, deadlines, and estimated hours and assign them within their authority scope.
- **Capacity Tracking and Overload Detection:** The system shall calculate assigned hours vs. available hours for lecturers and warn/flag overload conditions before and after assignment.
- **Cross-Boundary Request Workflows:** The system shall support cross-department and cross-faculty requests when an assigner needs a lecturer outside their scope, including approval/rejection and status tracking.
- **Progress Tracking:** Lecturers shall be able to update assignment progress (0–100%) with timestamped notes; supervisors shall view progress in dashboards.
- **Student Supervisor Requests:** Students shall be able to submit supervision requests; Deans shall review and assign a supervisor; students shall track request status.
- **Notifications and Audit Logging:** The system shall provide in-app notifications and maintain audit logs for critical actions and workflow events.
- **Workload appeals:** Lecturers shall be able to submit workload appeals with justification; Department Heads/Deans shall review and resolve them.
- **Role promotion workflows:** The system shall support controlled promotions (e.g., Lecturer → Department Head; Department Head → Dean) with appropriate approvals.
- **Assignment filtering and search:** Users and supervisors shall be able to search and filter assignments by status, priority, deadlines, department, and assignee.
- **Workload overview:** Deans and Department Heads shall view summarized capacity/overload information to support fair allocation decisions.

Non-Functional Requirements

- **Security:** Passwords shall be hashed; authentication shall use secure tokens; HTTPS shall be enforced; input validation and parameterized queries shall reduce injection risks.
- **Usability:** The interface shall be intuitive and role-specific, reflecting the university hierarchy and enabling key tasks with minimal steps.
- **Performance:** Dashboards should load quickly; APIs should use pagination and indexing to support large numbers of users and assignments.
- **Compatibility:** The system shall work on modern browsers (Chrome, Firefox, Safari, Edge) and be responsive across common screen sizes.
- **Maintainability:** The codebase shall be modular and documented to support future enhancements and maintenance.
- **Availability:** The system should be available at least **99.5%** of the time during working days, excluding scheduled maintenance, to support continuous office operations.
- **Recoverability (Backup & Restore):** The system should support automated database backups and allow restoration within a defined recovery time objective to prevent data loss.
- **Reliability:** The system should handle failures gracefully (e.g., meaningful error messages) and protect data consistency for task updates.

System Requirements (Hardware/Software)

Software Requirements

- Updated web browser (Chrome, Edge, Firefox, Safari)
- Supported operating system (Windows, macOS, Linux)
- Web server environment: Apache (recommended) or Nginx
- Server-side runtime: PHP 8.x with PDO extension enabled
- Database system: MySQL 8.x (or compatible MariaDB)
- Node.js 18+ and npm/pnpm (required for Next.js frontend build)

Software Requirements (Runtime/Platform)

- Package and dependency management: Composer (PHP) and npm/pnpm (JavaScript)

Development Tools

- Code editor / IDE: Visual Studio Code
- Version control: Git with GitHub (or GitLab)
- UI/UX design tool: Figma (wireframes, mockups, prototypes)
- API testing tool: Postman or Insomnia
- Database client/administration: phpMyAdmin, MySQL Workbench, or DBeaver
- Collaboration/communication (optional): Microsoft Teams / Discord
- Documentation (optional): Microsoft Word / Google Docs

Hardware Requirements

- Stable internet connection
- Laptop/Desktop computer
- Minimum 4GB RAM (8GB recommended)
- Minimum 10GB free disk space
- Dual-core processor or higher

User Roles

System Admin

- Manage global configuration, users, and roles across all faculties
- Approve and confirm promotions (e.g., Department Head → Dean) where required
- View audit logs and maintain the overall system (including maintenance mode)

Dean

- View faculty-wide capacity and workload overview across departments
- Assign work to Department Heads and Lecturers within the faculty
- Review student supervisor requests and assign a supervisor
- Handle cross-faculty requests (approve/reject) and manage overload alerts

Department Head

- Assign work to lecturers within their department and monitor progress
- View department workload/capacity and receive overload notifications
- Submit cross-department requests when needing a lecturer outside the department
- Review workload appeals and escalates/coordinate with the Dean when needed

Lecturer

- View assigned work, deadlines, and personal workload/capacity summary
- Update assignment progress (0–100%) and add timestamped progress notes
- Submit work requests or workload appeals through formal workflows

Student

- Submit a formal request for a project/thesis/research supervisor
- Track the status of submitted requests (e.g., Pending / Approved / Rejected / Assigned)

CHAPTER 4: PROJECT PLAN

Gantt Chart

The project plan is organized according to the Waterfall phases and distributed across 15 weeks. The schedule below represents the expected timeline for key deliverables, aligned with standard project management planning practices.

For day-to-day tracking during development, a Kanban-style visual workflow board can be used to monitor task status and work-in progress.[6]

Task	Weeks 1–3	Weeks 4–6	Weeks 7–9	Weeks 10–12	Weeks 13–15
Requirements Gathering & Analysis	X				
System Design (Architecture, ERD, Wireframes)		X			
Database Setup & Implementation		X	X		
Backend Development (APIs, RBAC, Modules)			X	X	
Frontend Development (Dashboards, UI)				X	X
Deployment, Testing & Bug Fixing					X
Documentation	X	X	X	X	X
Final Report submission & presentation					X

Table 1: Project Plan (Gantt Chart)

References

- [1] Ian Sommerville. 2011. *Software Engineering*, 9th ed. Pearson Education, Boston, MA, USA.
- [2] W3Schools. 2025. *W3Schools*. Retrieved May 2025 from <https://www.w3schools.com>
- [3] Bootstrap. 2025. *Build fast, responsive sites with Bootstrap*. Retrieved May 2025 from <https://getbootstrap.com/>
- [4] Bootstrap. 2025. *Bootstrap v5 introduction*. Retrieved May 2025 from <https://getbootstrap.com/docs/5.0/getting-started/introduction/>
- [5] Bootstrap Icons. 2025. *Free, high-quality, open-source icon library*. Retrieved May 2025 from <https://icons.getbootstrap.com/>
- [6] Gantt.com. 2025. *How to make a Gantt chart*. Retrieved May 2025 from <https://www.gantt.com/creating-gantt-charts>
- [7] Wireframe.cc. 2025. *Wireframe prototyping tool*. Retrieved May 2025 from <https://wireframe.cc/>
- [8] Stack Exchange Inc. 2008–2025. *Stack Overflow*. Retrieved May 2025 from <https://stackoverflow.com/questions>
- [9] PHP Group. 2025. *PHP Data Objects*. Retrieved May 2025 from <https://www.php.net/manual/en/book.pdo.php>
- [10] MySQL. 2025. *MySQL 8.4 Reference Manual*. Retrieved May 2025 from <https://dev.mysql.com/doc/refman/8.4/en/>
- [11] University College London. 2025. *Workload Allocation Framework for Academic Staff*. Retrieved May 2025 from <https://www.ucl.ac.uk/human-resources/workload-allocation-framework-academic-staff>
- [12] SDGP.LK. 2026. *SDGP Project Page*. Retrieved May 2025 from <https://www.sdgp.lk/project/2ce4736d-57c4-402d-af8f-811fedecc158>